Manuscript draft: Acceptance as an Event (Acceptance as Operation)

Target journal: *Technology in Society* (Elsevier)
Language: English (Academic)
Status: Draft v0.4 (EN, Major Revision) — updated 2026-02-01
Note: Major-revision English draft aligned with `草稿_验收即事件_CN_v0.4.md`.

> *Writing constraint: **do not** mention any internal project/system name. Treat this as a standalone governance proposal.*

Working title

From Judgment to Responsibility Anchors: Operationalizing Accountability with Auditable Acceptance Events

Abstract

In AI-assisted systems, "acceptance" is often treated as a private judgment rather than an auditable operation, leading to responsibility diffusion and post-hoc blame shifting. This paper redefines acceptance as an **acceptance event**: a discrete transaction binding (i) an artifact, (ii) a versioned constraint set, and (iii) an evidence bundle digest, to (iv) an authorized sign-off at time *t*. The core novelty is treating acceptance as the primary responsibility-binding operation via explicit artifact-constraint-evidence binding. We propose a **constraint referencing requirement** ("no acceptance without citation"), requiring every acceptance to cite a registered constraint set. Crucially, acceptance events define responsibility boundaries, not monopolies—making sign-off auditable without collapsing accountability into single-point blame. We illustrate the approach with an AI-generated code vignette and derive organizational and regulatory recommendations. The proposal is normative: it operationalizes accountability while warning that auditability can be absorbed into compliance theater.

Keywords: acceptance event; accountability; governance semantics; auditability; constraint referencing; responsible technology

---

1. Introduction: why “trust” cannot anchor responsibility

“Trust” is an important social concept, but it is a weak engineering control primitive. When an AI-assisted system causes harm, questions like “did we trust it?” are not operationally actionable: they do not identify *which criteria were applied*, *who had authority*, *what evidence existed at the time*, or *what exactly was accepted*.

Meanwhile, modern production creates a governance gap:

- systems can generate changes faster than humans can read and review them;
- implementations can be frequently regenerated, making authorship and intent difficult to attribute;
- responsibility diffuses (“everyone touched it, therefore no one owns it”).

Empirical signals. Across high-impact domains—healthcare AI deployment, developmental automated driving, algorithmic content moderation, and data-driven credit scoring—secondary literature repeatedly shows a common post-incident dispute pattern: parties contest which criteria and evidence were treated as sufficient at the moment of acceptance (Wong et al., 2021; NTSB, 2019; Gorwa et al., 2020; Hurley & Adebayo, 2016). We include brief mini-cases in §4.5 to illustrate this recurring structure.

Responsible technology framing. In responsible technology, the key question is not whether a system is “trusted,” but whether consequences can be traced to an auditable responsibility anchor. The goal is not to make systems more “intelligent,” but to make responsibility operationally locatable under automation (Kroll et al., 2017; Diakopoulos, 2016; Raji et al., 2020).

Boundary, not monopoly. Acceptance events do not impose single-point blame. They define a responsibility boundary and a decomposable, auditable interface: *a responsibility boundary, not a responsibility monopoly*.

Thesis. Treating acceptance as a **responsibility-binding operation** (rather than a psychological judgment) is a necessary condition for accountable governance in high-velocity automated systems.

Contributions

This paper makes four contributions:

1. **Definition**: acceptance events as a minimal auditable unit binding artifact + constraint set + evidence digest + authorized agent at time *t*.
2. **Mechanism**: a constraint referencing requirement (“no acceptance without citation” of a registered constraint set).
3. **Vignette**: AI-generated code acceptance illustrating how responsibility shifts from authorship to sign-off.

4. **Recommendations**: organizational and regulatory guidance for responsible acceptance logging.

Scope and method (conceptual + normative)

This paper is a conceptual and normative governance proposal. The method is constructive: (i) operationalize "acceptance" as an auditable event, (ii) propose a constraint-binding institutional rule ("no acceptance without citation"), and (iii) derive design rationale from accountability and audit-society literature. Empirically, we use an illustrative vignette and brief mini-cases from secondary literature to show the structure of "acceptance gaps," without claiming causal attribution or empirical generalization. Empirical evaluation is deferred to future multi-case studies and deployment analyses (see §6.1).

Disambiguation (to prevent a common misreading). This paper does **not** propose a new DevOps best practice, a security control, or a software lifecycle method. It does not claim that governance can be reduced to pipelines, checklists, or signatures.

Instead, it intervenes at the level of **governance semantics**: what it means, institutionally, to *accept* an artifact. We treat acceptance-event logging as an implementation-agnostic **governance semantics layer** that binds responsibility to an explicit artifact–constraint–evidence citation at the moment of sign-off.

1.4 Intervening at the governance semantics layer: filling the acceptance gap

The governance-semantics gap. Existing interventions often fall into two categories: (i) technical mechanism integrity (tests, provenance/attestations, policy enforcement) and (ii) principle-level commitments (transparency, accountability). Between them sits a governance-semantics gap: when an organization decides to accept and deploy an artifact, what does that decision mean institutionally—what should it bind, and what record should it produce to make responsibility auditable? We argue that acceptance events—coupled with a constraint referencing requirement—fill this gap.

Operationalization (as used in this paper). By "operationalizing accountability," we mean translating the vague question "who is responsible?" into a repeatable institutional operation that produces queryable acceptance-time records: what was accepted, under which explicit constraints, on which evidence, by which authorized role, at what time. This does not settle whether the constraints were sufficient; it makes the responsibility-binding moment auditable.

---

2. From mental approval to an acceptance event

2.1 Definition

We define an **acceptance event** as a discrete institutional operation that produces an auditable record. A minimally deployable acceptance event should simultaneously bind:

- **Artifact**: the object being accepted (e.g., a build, model, rule set, configuration bundle), identified by an immutable digest.
- **Constraint set**: explicit acceptance criteria and boundaries (tests, scans, invariants, operating constraints), versioned and referenceable.
- **Evidence bundle digest**: a digest/commitment to the evidence bundle referenced at acceptance time (tests, scans, probes, etc.).
- **Authorized agent**: the role/entity permitted to accept within an institution's governance structure.
- **Timestamp + decision outcome**: time + outcome (accept / reject / exception).
- **Institutional sign-off reference**: a record that links the decision to authorization and a responsibility-bearing sign-off.

In summary, an acceptance event is a time-bound, authorized sign-off record that binds an artifact, a cited constraint set, and an evidence bundle digest into a queryable responsibility anchor—making it institutionally hard to deny which criteria and evidence governed the decision.

Terminology note. In software engineering, "artifact" may specifically mean build artifacts; here we use it broadly as "the object being accepted."

Epistemic humility. An evidence bundle digest is a minimum unit of traceability, not a guarantee of truth. Evidence and metrics can be selectively produced, reified into "numbers that satisfy audits," and optimized under compliance pressure. We therefore treat evidence bundles as queryable commitments rather than sufficient proof of safety or legitimacy (Porter, 1995; Lampland & Star, 2009).

Clarification. "Sign-off" denotes an institutional responsibility-bearing operation; it does not require any particular cryptographic signature scheme. The goal is accountability traceability, not cryptographic non-repudiation.

We use "event" rather than "state" to emphasize time-binding and the sign-off point: rollbacks/replacements should be captured as new events to preserve a responsibility timeline.

The key is not formalism, but **queryability**: auditors should be able to ask which constraint set was cited, which artifact digest was accepted, which evidence digest was referenced, and which authorized role signed.

2.2 Minimal record schema (implementation-agnostic)

An acceptance event should be technology-agnostic but operationally precise. A minimal record should bind:

- artifact identifier (name + version + digest)
- constraint set identifier (ID + version + digest)
- evidence bundle identifier (digest)
- authorized agent identity (role + identity reference)
- timestamp + decision outcome (accept/reject/exception)

Together, these elements create the explicit **artifact-constraint-evidence binding** and thereby a responsibility anchor.

Figure 1 (conceptual governance overview; implementation-agnostic). Artifact-constraint-evidence binding → acceptance event → responsibility anchor (see `submission/figure1_mermaid.txt`).

Example record (illustrative):

```
{
  "acceptance_event_id": "acc:2026-01-31T00:00:00Z:artifact:payments-
api:1.0.7",
  "timestamp_utc": "2026-01-31T00:00:00Z",
  "agent": {
    "role": "release_arbiter",
    "id": "user:team-lead-001"
  },
  "artifact": {
    "name": "payments-api",
    "version": "1.0.7",
    "digest": "sha256:..."
  },
  "constraints": {
    "constraints_id": "PAYMENTS-ACCEPTANCE-V1",
    "version": "1.2",
    "digest": "sha256:..."
  },
  "evidence": {
    "bundle_digest": "sha256:...",
    "summary": {
      "tests": "pass",
      "security_scan": "pass",
      "mutation_testing": "pass"
    }
  },
  "decision": "accept",
  "signature": {
    "scheme": "institutional",
    "bundle_digest": "sha256:..."
```

```
    }
  }
```

2.3 What acceptance events are (and are not)

Acceptance events are	Acceptance events are not
A minimal auditable unit that binds an artifact, a cited constraint set, and an evidence digest to an authorized sign-off at time *t*.	A private mental judgment ("it seems fine") or an informal consensus.
A governance interface that makes acceptance decisions queryable and replayable.	Generic decision logging that only records "a decision happened" (Singh et al., 2018).
A traceability requirement for criteria and evidence at acceptance time.	Explainability or transparency into how an AI produced an output.
A complement to supply-chain mechanisms (provenance, attestations, policy-as-code) by adding governance semantics.	A replacement for those mechanisms.
A responsibility anchor that can support ethical deliberation by making trade-offs and authorization legible.	A substitute for ethical deliberation, or "more paperwork" for its own sake.

2.4 Distinguishing acceptance from review and certification

To avoid being read as "DevOps best practice" or "compliance again," we draw an operational boundary:

	Concept	Focus	Who is responsible / who signs	Does it anchor responsibility?
	---	---	---	---
	Code review	implementation quality (reliability/maintainability)	developers / reviewers (Bacchelli & Bird, 2013)	weak: consequences are hard to anchor to a stable record
	Certification / compliance	conformance to external standards	third parties / compliance function	partial: may not bind to a specific artifact and evidence at time *t*
	Acceptance event	accepted *under this constraint set* by *this authorized role* at *this time*	authorized acceptance role	yes: creates a queryable responsibility anchor

2.5 Comparison with supply-chain and governance practices (SLSA / attestation / policy-as-code)

A common concern is whether acceptance events are "just another name" for sign-off, release gates, attestations, or policy-as-code. Our answer is: we do not claim to invent a new signing technology or add a new "process gate." Instead, we propose a **governance semantics layer** that treats the acceptance decision itself as the primary responsibility-

binding operation, and requires it to be accountable for an explicit artifact-constraint-evidence binding.

The table below contrasts common practices (non-exhaustive):

Practice / technology	Primary question answered	Primary binding object	Typical output	Boundary / gap (from this paper's view)
SLSA provenance	"How was it built?"	build process, dependencies, artifact	build provenance	strong on process integrity; weak on governance semantics: does not necessarily answer "who accepted it under which criteria" (OpenSSF, 2023)
in-toto / attestations	"Which steps happened? who asserted what metadata?"	step metadata, inputs/outputs, signer	link metadata / attestations	strong on verifiable claims; weak on institutional meaning: a claim is not the same as an organization accepting consequences under a constraint set (Torres-Arias et al., 2019)
Sigstore/cosign, signed SBOM	"Who cryptographically signed this artifact/metadata?"	artifact or metadata (e.g., SBOM)	signature / signed SBOM	strong on integrity and identity; weak on responsibility semantics: a signature is not necessarily acceptance under explicit constraints (Newman et al., 2022)
Policy-as-code (e.g., OPA / Kyverno)	"Does it satisfy executable policy?"	policy rules and enforcement result	policy decision / admission decision	strong on automation; weak on traceable exceptions/rollbacks: who authorized exceptions, on what evidence, for how long

Practice / technology	Primary question answered	Primary binding object	Typical output	Boundary / gap (from this paper's view)
Acceptance event (this paper)	"Who accepted which artifact under which constraints, citing which evidence?"	artifact digest + constraint reference + evidence digest + authorized sign-off	acceptance-event record	directly anchors responsibility; can reuse the above mechanisms as inputs/evidence, but contributes a responsibility-binding governance meaning

Acceptance events should therefore be read as a **governance complement** to SLSA/attestations/policy-as-code: they do not replace their security/integrity functions, but fill the semantic gap of "how acceptance decisions bind criteria and responsibility."

Unlike supply-chain mechanisms that focus on *process integrity* (how it was built) or *assertion integrity* (who signed what metadata), we focus on the institutional meaning of *acceptance* as responsibility anchoring.
Where transparency or explainability work asks how systems can be understood, we ask how acceptance decisions can be made **auditable and contestable**: which constraints were cited, what evidence was referenced, and who was authorized to accept.

2.6 Minimum integration path and incremental cost (if SLSA / in-toto already exist)

If an organization already produces provenance and attestations (e.g., SLSA provenance, in-toto links), adopting acceptance events is primarily an **integration and governance** task rather than a new security mechanism.

Minimum integration path (thin layer):

1. **Register constraint sets** as versioned, referenceable objects (e.g., a Git directory of `constraints/` with stable IDs and versions). Treat them as first-class governance artifacts.
2. **Reuse existing pipeline outputs as evidence inputs**: tests, scans, provenance, policy decisions, and their existing attestations can be referenced and summarized into an evidence bundle digest.
3. **Emit an acceptance-event record at release time**: when a release gate would normally "approve," an authorized role produces an acceptance event that binds artifact digest + cited constraint set version + evidence digest + decision outcome + timestamp.
4. **Store acceptance events append-only** (e.g., an append-only log/ledger, immutable storage, or Git with protected history) and audit by query.

Incremental cost profile:

- **Engineering cost (often small):** compute/store digests and pointers; standardize schema; integrate log storage.
- **Governance cost (the real cost):** maintaining constraint sets (ownership, review cadence), operating exception/TTL rules, and running reproducibility-focused audits.

Having defined acceptance events, a key question follows: how do we ensure that acceptances cite stable, auditable criteria rather than implicit judgments? This motivates the constraint referencing requirement in §3.

---

3. Constraint referencing requirement: no acceptance without citation

3.0 Term clarification (referencing ≠ rigidity)

In some contexts, labels like "fixation" may suggest belief rigidity or institutional path dependence. Here, we use **constraint referencing requirement** to mean a *procedural requirement at acceptance time*: the acceptance event must explicitly cite and bind the **version and digest** of a registered constraint set, so criteria cannot silently drift post hoc.

This is an **explicit constraint-binding requirement**: acceptance is invalid unless it cites a registered constraint set.

Many governance failures are caused not by malice, but by **implicit constraints**: accepting artifacts because they "look reasonable" or "worked last time." Such criteria are unstable and unauditable.

We propose a rule:

> ***Every acceptance event must cite a registered constraint set.***

3.1 Registered constraint sets

A constraint set can be lightweight (a checklist + required evidence + risk thresholds), but it must be:

- explicit
- versioned
- referenceable
- stable enough to support audit queries

This paper does not evaluate whether the constraint set is "good enough"; that question belongs to governance/regulation. The point of the constraint referencing requirement is not to guarantee correct standards, but to ensure an organization cannot deny *which* standards it actually used after an incident.

3.2 Why the requirement works

The constraint referencing requirement suppresses three common failure modes:

1. **Post-hoc rationalization**: inventing criteria after an incident.
2. **Responsibility diffusion**: "everyone assumed someone else checked it."
3. **Criteria drift**: silently changing what "acceptable" means without leaving a trace.

3.3 Failure mode example: implicit constraint trap

If a team accepts an AI-generated change because "it looks correct," there is no constraint object to audit, no evidence digest to query, and no stable sign-off point. When incidents occur, investigations collapse into narrative debates rather than evidence-based accountability.

3.4 Critical reflexivity: when "auditability" becomes "tick-box compliance"

Eventizing acceptance and requiring constraint citations can surface implicit criteria. But we must acknowledge a second-order risk: **auditability can itself fail**—by incentivizing formalism, ritualistic compliance, and decoupling between documented standards and real work.

Institutional theory has long warned that organizations often build visible procedures and paperwork to signal control and legitimacy; these formal structures can drift away from actual practice and become symbolic "myths and ceremonies" (Meyer & Rowan, 1977). When constraint sets are reified into checklists, KPIs, or automated gates, teams can learn to optimize what is measurable and auditable rather than what is substantively safe; evidence bundles can be strategically produced to satisfy audit questions rather than to reveal risk (Porter, 1995; Lampland & Star, 2009).

Moreover, once constraint sets become institutionalized objects, they may be taken for granted, depoliticizing contestable judgments (Zucker, 1977) and accelerating isomorphic copying of templates across organizations (DiMaggio & Powell, 1983). In that situation, "no acceptance without citation" is highly prone to degenerating into "citation is sufficient": it compresses responsibility into a single sign-off action, obscuring rather than revealing power questions—who defines standards, who bears costs, and who can raise dissent (Scott, 2014).

We therefore treat the constraint referencing requirement as **necessary but not sufficient**, and recommend anti-formalism safeguards such as:

1. **Eventize constraint-set creation and changes**: record proposers, approvers, scope, and rationale; raise the bar for removing critical constraints.

2. **Bounded exceptions**: exceptions must cite a higher-level registered protocol, include TTLs, and impose remediation obligations.
3. **Audit reproducibility, not only documentation**: sample-and-replay audits and adversarial/chaos exercises to check evidence–decision consistency.
4. **Keep contestation channels visible**: record trade-offs and escalation paths within constraint sets.
5. **Do not fully automate acceptance**: systems may generate candidate evidence summaries, but "acceptance" as responsibility-binding must remain under meaningful human control (see §6.2.5).

---

4. Vignette: accepting AI-generated code under explicit constraints (expanded illustration)

This vignette is fictional but practice-oriented. It shows how acceptance events and a constraint referencing requirement operate under realistic organizational constraints (speed, division of labor, exceptions, and rollback windows).

Figure 2 (conceptual process comparison; governance-level). Traditional review-as-control vs acceptance-event control (see `submission/figure2_mermaid.txt`).

Figure 3 (conceptual decision logic; implementation-agnostic). When to accept, reject, or issue bounded exceptions with TTL and remediation (see `submission/figure3_mermaid.txt`).

4.1 Scenario: high-impact system + high-frequency change + AI-assisted generation

Consider a mid-sized fintech organization (pseudonym) operating a payments aggregation platform. The "charge/refund/reconciliation" chain spans multiple microservices, and deployments occur frequently in small increments. The organization faces dual pressure: shipping quickly for business needs while meeting reliability and security expectations typical of high-impact domains.

Historically, changes were written by developers, reviewed by peers, and released after tests and security checks. With AI coding assistants, the production cadence accelerates:

- a change may be specified by a human intention, generated by AI, and then locally edited by a human;
- the diff can be small while semantic impact is large (retry logic, exception paths, idempotency handling);
- when something goes wrong, "who wrote it" becomes harder to answer, while "who accepted what under which criteria" becomes more governance-relevant.

4.2 Traditional approach (review-as-control): bandwidth collapse and rubber-stamping

Under the traditional control model, responsibility is implicitly tied to "authorship + code review + tests passed."

But in AI-assisted contexts, teams are pulled toward two opposite but equally dangerous outcomes (Parasuraman & Riley, 1997):

- **Rubber-stamping**: reviewers rely on "the pipeline is green" and intuition, without a stable, auditable basis for whether key invariants and operational boundaries were actually checked.
- **Paralysis**: reviewers demand full semantic understanding of complex changes, which becomes unrealistic under high complexity and frequent regeneration; shipping then happens through informal bypasses.

Both outcomes share the same governance gap: even if someone "felt" that enough review happened, the organization cannot reliably answer auditable questions after an incident. What criteria were in force at the time? Were exceptions used? If so, who authorized them, on what evidence, and with what time limit?

4.3 Acceptance-event approach (sign-off-as-control): cited standards + evidence digest + exception events

Under acceptance event governance, the organization first makes "acceptable" explicit as a registered constraint set, tiered by risk. Because the payments path is high-impact, releases must cite a constraint set such as `PAYMENTS-ACCEPTANCE-V1`.

A constraint set (illustrative) can include:

- **Security and quality thresholds**: static analysis for OWASP Top-10-relevant issues with 0 critical findings; no known critical dependency vulnerabilities; mutation testing score not below 80%.
- **Key invariants**: verifiable idempotency-key behavior; no double-charge; required observability fields for key ledger events.
- **Operational boundaries and rollback readiness**: timeouts and retry limits configured; circuit-breaking policy set; canary rollout configuration present; rollback strategy drilled or validated in a controlled exercise/chaos environment.

The pipeline generates an evidence bundle (test reports, scans, probes) and computes an evidence digest. An authorized role (e.g., release arbiter) then issues an acceptance event that binds:

- the exact artifact digest;
- the exact cited constraint set version;
- the evidence bundle digest;
- the signer role, time, and outcome (accept / reject / exception).

The key shift is that "acceptance" is no longer dispersed across conversations and informal agreement. It becomes a queryable responsibility anchor. When incidents occur, investigations start from the acceptance event rather than from narrative reconstructions:

- When and by whom was this artifact accepted?
- Which constraint set version was cited? Was it registered beforehand? Was it modified later?
- Is the evidence bundle reproducible? Are there missing or selectively presented materials?
- Was an exception issued? Under which authorization boundary, with which TTL and remediation obligations?

To make the example concrete, imagine the AI-generated change is framed as a "small reliability improvement": it refactors retry logic around a charge request and subtly shifts where idempotency keys are validated. The diff is short and tests are green, but under concurrency the change creates a timing window in which two retries can slip through and produce a double-charge.

When such an incident is discovered weeks later, the acceptance event becomes the investigation entry point. The organization can query: which exact artifact digest was accepted, under which constraint set version, with which evidence digest, and by which authorized role. Crucially, the log can also surface a second-order governance question: even if the team was compliant with the cited constraint set, was the constraint set itself adequate? For instance, the replay may reveal that the cited constraint set version did not explicitly require an idempotency invariant test under concurrent retries. That deficiency points responsibility not only to sign-off, but also to constraint authorship/approval and to evidence design. The response then becomes eventized learning: the organization updates the registered constraint set (new version + rationale), adds the missing invariant test and operational guardrails, and issues a new acceptance event for the remediated release.

Acceptance-event governance must also confront **exceptions and rollback windows**. Consider an operational incident where a hotfix must be released within 30 minutes to stop losses.

- the exception cites a higher-level pre-registered constraint set (e.g., `INCIDENT-RESPONSE-PROTOCOL-V1`);
- it records justification, TTL, and risk controls (e.g., canary to 5% traffic plus tightened monitoring thresholds);
- it records remediation obligations (e.g., evidence completion within N hours followed by a standard acceptance event).

Result. Acceptance events do not guarantee "no failures," but they change the governance epistemics of incidents. They provide **temporal anchoring** of responsibility (what was accepted, by whom, under which constraints, at time *t*) and **institutional non-deniability of criteria** (an organization cannot credibly deny which constraint set version governed the decision). As a result, accountability work can shift from narrative dispute toward **query-based audits** over acceptance logs, decomposing responsibility across auditable role chains (constraint authorship/approval, evidence production integrity, acceptance sign-off, exception approval, and operational response).

4.4 Historical analogy (footnote-only)

Similar "acceptance gaps" have appeared in discussions of safety-critical incidents[1]. We do not revisit factual disputes; we use these as a structural reminder that absent explicit criteria citation and queryable sign-off records, incidents revert to narrative argument.

4.5 Mini-cases from secondary literature (brief)

The cases below are illustrative and non-exhaustive; we use them to show the recurring structure of acceptance gaps across domains rather than to claim empirical generalization.

Healthcare AI deployment (sepsis prediction). External validation of a widely implemented proprietary sepsis prediction model reported substantial performance gaps in real-world hospital settings, raising governance questions about how deployment decisions were justified and what evidence was treated as sufficient (Wong et al., 2021). In acceptance-event terms, "accepting the model for clinical decision support" should cite a constraint set specifying performance and safety criteria (including subgroup performance, alert burden, and monitoring/rollback obligations) and bind the cited evidence bundle used at the time of adoption. Without such a record, accountability tends to diffuse across vendors, clinicians, and institutions, and post-incident debate collapses into competing narratives about what was "known" or "required."

Automated driving (developmental AV crash). Investigations into a fatal crash involving a developmental automated driving system highlighted safety-management and risk-control failures in testing operations (NTSB, 2019). Acceptance-event governance would treat "allowing on-road operation under an operational design domain" as an acceptance decision requiring explicit constraint citation (e.g., ODD limits, safety-driver readiness, disengagement thresholds, and incident-response protocols) and an auditable evidence digest (testing results, hazard analyses, and monitoring data). The point is not that acceptance events would mechanically prevent incidents, but that they would make responsibility boundaries and applied criteria queryable.

Content moderation (automated enforcement and ranking). Research on algorithmic content moderation highlights recurring opacity about how enforcement is operationalized, how moderation rules are updated, and how automation is used to scale platform governance (Gorwa et al., 2020). In acceptance-event terms, deploying or updating moderation models/policies should cite a constraint set describing boundary conditions (policy definitions, error tolerances, appeal/override procedures, monitoring obligations) and bind the evidence digest used to justify the change. Without such acceptance records, disputes about harms tend to devolve into contested narratives about what the platform "intended" or "knew," rather than what criteria governed acceptance.

Credit scoring / risk models (lending decisions). Work on data-driven credit scoring emphasizes opacity and accountability challenges, including discrimination risks and difficulties of contestation (Hurley & Adebayo, 2016). In acceptance-event terms, putting a scoring model into use should cite constraints specifying acceptable performance, fairness/compliance criteria, adverse-action requirements, monitoring, and rollback thresholds, and bind the evidence bundle used at deployment. Without this, accountability becomes diffuse across vendors, lenders, and regulators, and post-hoc review struggles to reconstruct which acceptance criteria were in force.

---

5. Practical and policy recommendations

5.1 Organizational recommendations (minimum viable implementation)

The proposal is not "re-validate everything for every change." It is: **whenever an artifact is accepted, an acceptance event must be produced.**

1. **Define authorization roles**: specify who can sign acceptance for each risk class.
2. **Make constraints versioned**: treat acceptance criteria as a first-class, versioned artifact.
3. **Require evidence bundles**: acceptance without evidence is invalid.
4. **Maintain an append-only acceptance log**: treat it as a governance ledger.
5. **Audit by query**: periodically sample accepted artifacts and verify evidence integrity and reproducibility.

5.2 Policy recommendations (for regulators and high-impact domains)

This proposal can be translated into regulatory requirements without prescribing a single technical stack, because it governs *acceptance operations* rather than implementation details.

5.2.1 Risk-tiered acceptance-event requirements

Regulators can require acceptance-event logging proportionate to system risk:

- **High-impact systems** (e.g., finance, healthcare, critical infrastructure): require (i) registered constraint sets, (ii) acceptance events for every production deployment and material model/rule/config update, and (iii) bounded exceptions with TTLs and remediation obligations.
- **Medium/low-risk systems**: allow sampling-based requirements (e.g., acceptance events for major releases or for a random subset), while still requiring a clear authorization policy.

5.2.2 Confidentiality-preserving verification (trade secrets and security sensitivity)

To avoid forcing full disclosure of internal standards, regulators can accept **hash commitments** for constraint sets and evidence bundles:

- publish verifiable digests (commitments) externally;
- disclose full contents only under controlled audit conditions (e.g., accredited auditors, secure rooms, time-bounded access).

This enables regulators to verify *which* standards were in force at the time of acceptance, without requiring those standards to be fully public.

5.2.3 Cross-border recognition and supply-chain interoperability

For cross-border systems and multi-party supply chains, a practical direction is to standardize a minimum acceptance-event schema (artifact digest + cited constraint set + evidence digest + authorized role + timestamp + decision/exception + TTL). Mutual recognition can then operate on:

- schema compatibility and audit-query capability;
- local equivalence mappings for what counts as "registered constraints" and "authorized roles," without assuming infrastructure-rich contexts.

5.2.4 Regulatory audit queries (operational examples)

A key advantage of acceptance-event logs is that compliance checks can be expressed as audit queries, for example:

- show deployments accepted without a cited constraint set;
- show exceptions above a threshold, or exceptions whose TTL expired without remediation;
- show acceptances with missing/irreproducible evidence bundles;
- trace who was authorized to accept, and whether authorization boundaries were exceeded.

These queries support accountability without collapsing governance into "paperwork-only" verification.

---

6. Discussion and limitations

6.1 Technical limitations and implementation risks

- **Evidence integrity**: acceptance logs are only as trustworthy as evidence generation. Evidence should be produced in controlled environments (e.g., trusted CI) to reduce spoofing (Souppaya et al., 2022).
- **Cost and friction**: acceptance events add overhead. The goal is to move overhead from continuous human review to structured evidence + concentrated sign-off.
- **Agility tension (continuous deployment).** Although this paper is not a DevOps prescription, any acceptance-logging regime risks being perceived as "slowing shipping." The intent is to relocate governance effort from fragile, continuous human review to repeatable evidence and explicit, time-bounded sign-off. Tiered constraint sets, automation, and eventized exceptions/rollbacks can preserve high-velocity delivery while keeping responsibility anchors auditable.
- **Revocation, rollback, and time windows**: emergency rollbacks may occur between an original acceptance event and a new acceptance/rollback event. The minimum requirement is that rollback/replacement decisions should also be eventized and bound to constraints/evidence, preserving a timeline and clarifying what risk controls and authorization applied during the "gap". In this sense, the system's runtime state during the window is itself an **accepted risk configuration**.
- **Empirical scope**: this paper is a conceptual and normative governance proposal. The vignette is illustrative rather than a claim of empirical generalization; future work should include multi-case studies and deployment evaluations of acceptance-event practices.

6.2 Governance limitations and ethical risks

This section emphasizes: acceptance eventization is not an automatic guarantee of "better governance." Its value is that it makes responsibility chains and contestable points queryable. But auditability can itself become a power technique and a formalistic device.

6.2.1 Responsibility justice and distributed responsibility: avoiding scapegoating signers

Acceptance events anchor responsibility at a sign-off point, but they should not be used to search for a single scapegoat. Many harms are produced by structural conditions and institutional arrangements rather than individual malice. Responsibility should therefore be understood as forward-looking obligations to repair and redesign institutions, not only backward-looking blame allocation (Young, 2011).

Practically, this paper argues for decomposing responsibility into auditable role chains rather than compressing it into one signature:

- **Constraint-set authorship/approval responsibility**: those who define "acceptable" boundaries must be accountable for the adequacy and evolution of those boundaries.
- **Evidence generation and integrity responsibility**: those who generate tests/scans/evaluations must be accountable for reproducibility and completeness.
- **Acceptance sign-off responsibility**: those who accept/issue exceptions must be accountable for the cited standards and evidence at the time.
- **Operations and rollback responsibility**: those who operate systems and manage rollback windows must be accountable for risk configurations during the window (see §6.1).

The goal is not to dilute responsibility, but to prevent governance from collapsing into "punish the signer" while leaving institutions unchanged.

6.2.2 The shadow of auditability

Audit-society research warns that audits and metrics reshape behavior: they can incentivize strategic documentation, metric gaming, and compliance theater. When governance becomes synonymous with "being able to produce evidence," organizations can become better at producing presentable artifacts than at surfacing real risk (Power, 1997; Strathern, 2000; Shore & Wright, 2015).

Acceptance events can amplify this tendency: teams optimize "good-looking evidence digests," constraint sets drift into easy-to-pass templates, and exceptions become routine bypasses. The constraint referencing requirement should therefore be paired with anti-gaming practices such as sample-and-replay audits, exception-frequency reviews, and making evidence-production labor visible in governance deliberation.

6.2.3 Relation to supply-chain mechanisms: a governance semantics layer, not a replacement

As §2.5 clarifies, provenance answers "how it was built," attestations/signing answer "who asserted what," and policy-as-code answers "does it satisfy executable policy." Acceptance events answer "who accepted what under which constraints, citing which evidence," thereby adding a responsibility-binding semantics layer. They do not replace in-toto/SLSA/Sigstore; they complement them (Torres-Arias et al., 2019; OpenSSF, 2023; Newman et al., 2022).

6.2.4 Power, labor, and political economy (including the Global South): who can afford auditability

Auditability is not neutral. It assumes infrastructure (logging, reproducible builds, scanners), specialized staff, and ongoing maintenance. In practice, these costs can be shifted onto frontline engineers, contractors, or downstream suppliers; audit artifacts can also become instruments of surveillance and performance control (Zuboff, 2019),

reinforcing asymmetries under platformization and outsourcing (Srnicek, 2017). In cross-organizational and cross-border supply chains, auditability requirements can become compliance barriers that exclude resource-constrained actors, producing governance imperialism or data-colonial dynamics (Couldry & Mejias, 2019).

For this reason, we reject a context-free reading of "no acceptance without citation." In resource-constrained settings, tiered constraints, minimum viable evidence, and audit support mechanisms are required—otherwise the constraint referencing requirement can degrade from a responsibility anchor into an exclusion mechanism.

6.2.5 Against automated acceptance: meaningful human control

Finally, acceptance events can be supported by automation (e.g., auto-summarizing evidence and computing digests), but "acceptance/exception acceptance" as a responsibility-binding act should not be fully automated. Acceptance involves irreducible normative trade-offs (risk, benefits, rights, distribution). Delegating acceptance to automated gates or model scores hides contestation inside thresholds and can amplify automation bias and rubber-stamping (Parasuraman & Riley, 1997). We therefore argue that high-impact deployments must preserve meaningful human control, keeping visible who made which trade-offs under which authorization.

We operationalize meaningful human control in this framework as at least:

- **Constraint-set governance**: humans approve and revise the constraint sets that govern acceptance.
- **Exception governance**: humans deliberate and sign bounded exceptions (with justification, TTL, and remediation obligations).
- **Interpretive discretion over evidence**: humans retain authority to interpret and contest automated evidence summaries and metrics, rather than treating them as decisive.

In summary, acceptance events can strengthen accountability only when embedded in institutional arrangements that (i) distribute responsibility across auditable role chains, (ii) resist audit-theater incentives, (iii) remain interoperable with supply-chain mechanisms as a governance semantics layer, and (iv) account for power, labor, and resource asymmetries in cross-border contexts. Otherwise, "no acceptance without citation" risks becoming a ritual of verification, a liability-shifting technique, or an exclusionary compliance barrier.

---

7. Conclusion

This paper proposes a governance primitive: redefining "acceptance" as an auditable **acceptance event** and enforcing a **constraint referencing requirement** ("no acceptance without citation"), so each acceptance decision explicitly binds an artifact, a cited constraint set, and an evidence bundle digest. By fixing acceptances in queryable records, acceptance events provide **temporal anchoring** of responsibility (what was

accepted, by whom, under which constraints, at time *t*) and **institutional non-deniability of criteria** (organizations cannot credibly deny which constraint set governed the decision). This shifts accountability work from narrative dispute toward query-based audits over acceptance logs.

We also emphasize limits. Acceptance eventization does not automatically improve safety or legitimacy; it can be absorbed into audit culture as compliance theater (§3.4, §6.2.2), and it can be weaponized to shift liability onto frontline signers or to exclude resource-constrained actors (§6.2.1, §6.2.4). We therefore frame acceptance events as a **minimum queryable interface** that must be coupled with responsibility justice, organizational checks on power, and continuous post-incident learning.

Ethically, we reject two common misuses:

1. Treating "having records" as "having responsibility," enabling compliance without accountability.
2. Delegating acceptance to automated gates or model scores and eroding meaningful human control.

For high-impact deployments, acceptance must remain attributable to accountable human decision-makers, with contestation and trade-offs kept visible and auditable rather than buried in thresholds and workflows. This is necessary structure, not rigid restraint: the goal is to keep criteria and trade-offs visible, queryable, and contestable.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used Google Gemini in order to assist with English translation, language editing, and formatting. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

---

8. References (candidates; to be formatted per JRT)

1. Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE 2013)*, 712–721. DOI: 10.1109/ICSE.2013.6606617.
2. Diakopoulos, N. (2016). Accountability in algorithmic decision making. *Communications of the ACM*, 59(2), 56–62. DOI: 10.1145/2844110.
3. Kroll, J. A., Huey, J., Barocas, S., Felten, E. W., Reidenberg, J. R., Robinson, D. G., & Yu, H. (2017). Accountable algorithms. *University of Pennsylvania Law Review*, 165(3), 633–705.
4. Parasuraman, R., & Riley, V. (1997). Humans and automation: use, misuse, disuse, abuse. *Human Factors*, 39(2), 230–253. DOI: 10.1518/001872097778543886.

5. Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Barnes, P. (2020). Closing the AI accountability gap: defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT 2020)**. arXiv:2001.00973.
6. Singh, J., Cobbe, J., & Norval, C. (2018). Decision provenance: harnessing data flow for accountable systems. arXiv:1804.05741.
7. Souppaya, M., Scarfone, K., & Dodson, D. (2022). Secure software development framework (SSDF) version 1.1. *NIST Special Publication 800-218*. DOI: 10.6028/NIST.SP.800-218.
8. Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., & Cappos, J. (2019). in-toto: providing farm-to-table guarantees for bits and bytes. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security 19)*, 1393–1410.
9. OpenSSF. (2023). Supply-chain Levels for Software Artifacts (SLSA) v1.0.
10. Newman, Z., Meyers, J. S., & Torres-Arias, S. (2022). Sigstore: software signing for everybody. In *Proceedings of the 2022 ACM Conference on Computer and Communications Security (CCS 2022)*.
11. DiMaggio, P. J., & Powell, W. W. (1983). The iron cage revisited: institutional isomorphism and collective rationality in organizational fields. *American Sociological Review*, 48(2), 147–160.
12. Scott, W. R. (2014). *Institutions and Organizations: Ideas, Interests, and Identities* (4th ed.). Sage.
13. Power, M. (1997). *The Audit Society: Rituals of Verification*. Oxford University Press.
14. Young, I. M. (2011). *Responsibility for Justice*. Oxford University Press.
15. Akrich, M. (1992). The de-scription of technical objects. In W. E. Bijker & J. Law (Eds.), *Shaping Technology/Building Society: Studies in Sociotechnical Change* (pp. 205–224). MIT Press.
16. Winner, L. (1980). Do artifacts have politics? *Daedalus*, 109(1), 121–136.
17. Meyer, J. W., & Rowan, B. (1977). Institutionalized organizations: formal structure as myth and ceremony. *American Journal of Sociology*, 83(2), 340–363. DOI: 10.1086/226550.
18. Zucker, L. G. (1977). The role of institutionalization in cultural persistence. *American Sociological Review*, 42(5), 726–743. DOI: 10.2307/2094862.
19. Porter, T. M. (1995). *Trust in Numbers: The Pursuit of Objectivity in Science and Public Life*. Princeton University Press.
20. Lampland, M., & Star, S. L. (Eds.). (2009). *Standards and Their Stories: How Quantifying, Classifying, and Formalizing Practices Shape Everyday Life*. Cornell University Press.
21. Strathern, M. (Ed.). (2000). *Audit Cultures: Anthropological Studies in Accountability, Ethics and the Academy*. Routledge.

22. Shore, C., & Wright, S. (2015). Audit culture revisited: rankings, ratings and the reassembling of society. *Current Anthropology*, 56(3), 421–444.
23. Zuboff, S. (2019). *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. PublicAffairs.
24. Srnicek, N. (2017). *Platform Capitalism*. Polity.
25. Couldry, N., & Mejias, U. A. (2019). *The Costs of Connection: How Data Is Colonizing Human Life and Appropriating It for Capitalism*. Stanford University Press.
26. National Transportation Safety Board (NTSB). (2019). *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. Accident Report NTSB/HAR-19/03.
27. Wong, A., et al. (2021). External validation of a widely implemented proprietary sepsis prediction model in hospitalized patients. *JAMA Internal Medicine*.
28. Gorwa, R., Binns, R., & Katzenbach, C. (2020). Algorithmic content moderation: Technical and political challenges in the automation of platform governance. *Big Data & Society*, 7(1).
29. Hurley, M., & Adebayo, J. (2016). Credit scoring in the era of big data. *Yale Journal of Law and Technology*, 18, 148–216.

---

1. Commonly discussed examples include Therac-25 and Boeing 737 MAX (MCAS). We do not analyze details here; we only reference the structure "acceptance gap → narrative dispute." ↩